

Введение

В эпоху, когда машинным обучением не занимается только ленивый... Пожалуй, такая формулировка не будет соответствовать реальности. В реальности многие инструменты машинного обучения ориентированы как раз на «ленивого» пользователя: это и полностью сконфигурированные предварительно обученные нейросети, и инструменты для автоматического подбора параметров, и даже инструменты для автоматического выбора метода обучения (модели). Поэтому точнее будет сформулировать тезис следующим образом. В условиях, когда машинным обучением занимается даже ленивый, нет необходимости обосновывать актуальность этого научного направления, поскольку она очевидна.

Возможно, текущие ожидания в этой области даже несколько (а может быть и значительно) завышены. Однако реальная полезность данных инструментов не вызывает сомнений, равно как и их востребованность в обозримом будущем.

Следует уточнить терминологию. Мы будем использовать как синонимы следующие термины: машинное обучение, интеллектуальный анализ данных, теория статистических решений, data mining, распознавание образов.

Также будем использовать широкий термин «искусственный интеллект», относя к этой области любые вопросы, связанные автоматизацией интеллектуальной деятельности, т.е. с передачей компьютеру интеллектуальных функций человека.

Представленный материал в первую очередь относится к машинному обучению, но некоторые вопросы будут касаться как раз более широкой области.

Можно выделить два подхода к использованию машинного обучения, которые условно назовём «инженерным» и «исследовательским».

Первый подход предполагает использование готовых «рецептов» для построения моделей машинного обучения. Для этого требуется широкий кругозор, изучение последних актуальных инструментов и знание большого числа шаблонов для решения разных практических задач. При этом не требуется глубокого понимания используемых методов и приёмов. По большому счёту достичь такого понимания для всех актуальных инструментов физически нереально ввиду их большого количества и постоянного обновления. Подобная квалификация широко востребована. Вместе с тем, такие специалисты испытывают затруднения при решении нестандартных задач, требующих доработки или модификации известных методов.

Чтобы решать нестандартные задачи или разрабатывать новые методы анализа данных, требуется хорошо разобраться в их принципах и тонкостях работы. При этом не получится ограничиться только одной областью машинного обучения (например, только нейросетевыми методами). Нейронные сети являются основным инструментом при решении многих задач анализа данных (хотя и далеко не всех, например, на табличных данных нейросети, как правило, уступают бустингу), но инструментом, сложным для анализа и неподходящим для изучения принципов машинного обучения. Чтобы научиться понимать сложные нейросетевые модели, целесообразно потренироваться на более простых моделях.

Поэтому если ставить целью получение не только инженерной, но и исследовательской квалификации, то целесообразно системно изучать основные методы и подходы машинного обучения, от простых к более сложным. Именно такую цель преследует данное учебное пособие. Большое внимание будет уделено выявлению неочевидных связей между различными методами, а также фундаментальным проблемам машинного обучения. Необходимо разобраться с математическим фундаментом, на котором основывается область искусственного интеллекта.

Следует отметить отличия данного учебника от классических

курсов (например, [1,2]), а также большого количества открытых курсов и пособий по машинному обучению.

Во-первых, существенно отличается структура и изложение материала. Это связано с тем, что в анализе данных существуют разные научные школы, и если математические факты для всех одинаковы, то их интерпретация и практика применения в разных школах могут различаться. Разумеется, автор не считает «своё» изложение более «правильным» по сравнению с другими. Материал излагается так, как было бы понятнее самому автору. Предполагается, что читателю в любом случае, полезно рассмотреть материал с разных точек зрения.

Во-вторых, здесь, несмотря на относительно небольшой объём, есть материал, который редко встречается в других источниках.

В-третьих, пособие предполагает широкую целевую аудиторию и предназначено как для начального изучения предмета, так и для специалистов по анализу данных. Такая универсальность достигается за счёт того, что темы выбираются не по их сложности, а по важности.

Рассматриваются разные вопросы, среди которых есть как простые, так и сложные. При этом для сложных вопросов предварительно даётся некоторое краткое изложение основных идей, так чтобы при первом чтении можно было пропустить сложные детали и это не помешало пониманию дальнейшего материала.

Автор будет рад получить комментарии и замечания (на адрес: nedelko@math.nsc.ru).

На первый взгляд, в математике не может быть разных точек зрения, поскольку если теорема доказана, то уже не получится с ней не согласиться. Но разные точки зрения могут быть в интерпретации утверждений. Например, теорема Гёделя о неполноте. Иногда её интерпретируют как довод в пользу того, что в математике существуют утверждения, которые нельзя ни доказать, ни опровергнуть. Но из самой теоремы подобного не следует, это всего лишь интуитивный довод на основе аналогии.

В машинном обучении очень много подобных примеров, когда некоторые строгие утверждения используются для аргументации некоторых гипотез, которые лишь косвенно связаны с доказанными

ми утверждениями.

В данном материале мы будем особо акцентировать внимание на таких «общеизвестных фактах», которые на самом деле не доказаны и, вполне вероятно, фактами не являются.

Особо отметим, что это пособие не является монографией. Если в монографии результат приводится без ссылок, то это значит, что результат принадлежит автору. Если в учебнике результат приводится без ссылки — значит автор считает результат общеизвестным.

В этом пособии именно так: все собственные результаты автора приведены со ссылкой; всё, что дано без ссылки — автор считает общеизвестным или слишком простым, чтобы устанавливать авторство.

Автор настоятельно рекомендует (до или после чтения данного пособия) ознакомиться с [1]. Это классический курс по машинному обучению. Включает конспекты лекций, презентации и видеозаписи лекций. Кроме того, на ресурсе `machinelearning` имеется много полезных статей данного автора, в том числе рекомендации по ведению научной работы для студентов.

Также рекомендуется (попытаться) изучить учебник [2]. Но это только в качестве следующего шага, т.к. он труднее для понимания.

Для дальнейшего чтения можно порекомендовать также [3]. Статьи этого автора (который длительное время имел самый высокий рейтинг на `kaggle`) полезны как источник практических рекомендаций и приёмов для решения реальных задач.

При этом автор настоятельно не рекомендует начинать изучение предмета с `online`-учебников и интернет-курсов. Они зачастую представляют собой набор статей разных авторов, поэтому по ним трудно получить цельное представление о материале. Также в подобных источниках многие вопросы освещаются поверхностно.

В текущих реалиях невозможно представить себе эффективное обучение чему-либо без использования интернет-ресурсов.

Для изучения предмета автор рекомендует использовать следующие источники.

<http://kaggle.com/> — ресурс, который нужно использовать в обя-

зательном порядке. Первоначально это была просто платформа для проведения конкурсов по анализу данных. Однако очень быстро стал понятен её образовательный потенциал, а также значение как «полигона» для сравнения различных методов анализа данных, который позволяет оценить их реальную полезность. Главную образовательную ценность на этом ресурсе имеют законченные примеры решения реальных задач лучшими специалистами, представленные в виде полностью рабочего кода с пояснениями.

<http://archive.ics.uci.edu/ml/datasets.html> — UCI (University of California, Irvine) Machine Learning Repository. Наиболее известная (до появления kaggle) коллекция задач машинного обучения, содержащая несколько сотен наборов данных из различных прикладных задач. На задачи этого репозитория принято ссылаться в академических публикациях, где они используются в качестве реальных данных. Использование одинаковых данных разными исследователями делает получаемые результаты воспроизводимыми.

<http://www.machinelearning.ru/> — информационный русскоязычный ресурс по анализу данных. Содержит статьи разного уровня по качеству (важно смотреть на авторство статьи), но большое число статей полезно в качестве источника справочной информации.

<https://education.yandex.ru/handbook/ml> — достаточно качественный справочник по машинному обучению. Категорически не рекомендуется для первоначального знакомства с предметом, однако полезен для дополнительного обучения.

В названии курса говорится о машинном обучении в вероятностной постановке. Нужно оговорить, что такое название вовсе не означает каких-либо ограничений на круг рассматриваемых вопросов и методов. Мы будем рассматривать в том числе и невероятностные методы и подходы. А вероятностная постановка упомянута скорее как намерение по возможности использовать вероятностную интерпретацию задач и результатов. Иными словами,

речь будет идти о «самом обычном» машинном обучении.

Глава 1

Метод прецедентов

Метод прецедентов является, пожалуй, наиболее простым методом машинного обучения и наиболее интуитивным. Фактически он отражает естественную человеческую стратегию поведения: для принятия решения в новой ситуации найти похожие случаи из прошлого опыта и действовать по аналогии.

Метод прецедентов является на самом деле не конкретным методом, а некоторым подходом (классом методов), в рамках которого существует ряд методов, по-разному реализующих идею прецедентов или аналогии.

Для объекта, который нужно классифицировать, находятся наиболее похожие на него объекты, для которых целевой признак известен.

Прогноз осуществляется на основе «голосования» по прецедентам.

Данный класс методов также называют метрическими методами построения решающих функций. При этом методы вовсе не требуют, чтобы функция (не)похожести объектов была именно метрикой.

1.1 Метод ближайшего соседа в задаче классификации

Для знакомства с методом прецедентов рассмотрим одну простую практическую задачу машинного обучения, а именно задачу «Iris» из репозитория UCI.

Задача состоит в том, чтобы построить правило, позволяющее по внешним признакам различать три вида (сорта) ирисов (цветы):

1 – Iris Setosa,

2 – Iris Versicolour,

3 – Iris Virginica.

Вид ириса обозначим переменной Y , которая будет принимать значения 1, 2, 3.

В качестве измеряемых характеристик используются:

X_1 – длина чашелистика (sepal length),

X_2 – длина лепестка (petal length).

Таким образом, требуется по измеренным значениям переменных X_1 и X_2 определить вид ириса.

Результаты измерений указанных характеристик для 36 экземпляров растений трёх видов приведены в таблице 1.1. Здесь i обозначает номер объекта (экземпляра) и в верхней позиции обозначает не степень, а верхний индекс, x_1^i – значения переменной X_1 , x_2^i – значения переменной X_2 , y^i – значения переменной Y .

Заметим, что в исходной таблице данных, доступной в репозитории, используются четыре характеристики, а число объектов составляет 150, однако, для наглядности мы рассматриваем сокращённую таблицу.

Для наглядности изобразим имеющиеся данные на диаграмме (см. рис. 1.2). Требуется, проанализировав представленные данные (будем называть их выборкой), научиться распознавать сорт

i	x_1^i	x_2^i	y^i	i	x_1^i	x_2^i	y^i
1	5,1	1,4	1	13	6	4	2
2	4,9	1,4	1	14	6,1	4,7	2
3	4,7	1,3	1	15	5,6	3,6	2
4	4,6	1,5	1	25	6,7	5,7	3
5	5	1,4	1	26	7,2	6	3
6	5,4	1,7	1	27	6,2	4,8	3
7	4,6	1,4	1	25	6,7	5,7	3

Рис. 1.1: Данные по ирисам.

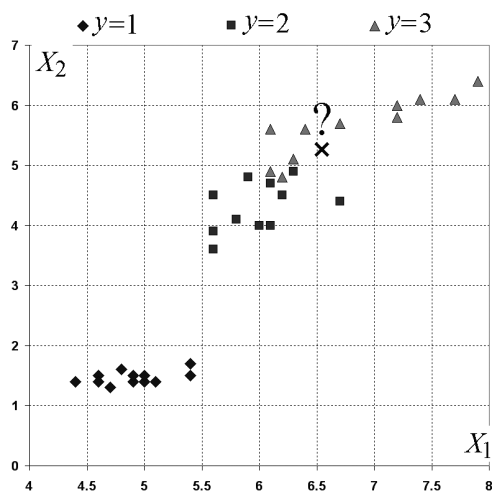


Рис. 1.2: Визуализация данных из таблицы 1.1.

ириса по внешним признакам (X_1 и X_2). Математически это означает, что необходимо построить функцию f , сопоставляющую любой паре (x_1, x_2) значений переменных X_1, X_2 некоторое значение y переменной Y .

Такая функция называется решающей функцией или решающим правилом:

$$f : X_1 \times X_2 \rightarrow Y$$

или $y = f(x_1, x_2)$, $y \in Y$, $x_1 \in X_1$, $x_2 \in X_2$. Здесь и далее мы обозначаем одинаково переменную и множество значений, которые она принимает.

Идея метод прецедентов заключается в следующем. Для объекта, который нужно классифицировать, находятся наиболее похожие на него объекты, для которых целевой признак известен.

Простейшая модификация метода заключается в том, что распознаваемый объект относится к тому же классу, к которому принадлежит объект, на который он более всего похож. Такой метод носит название метода ближайшего соседа.

В качестве «меры (не)похожести» можно использовать расстояние ρ между точками, представляющими объекты в признаковом пространстве. Так например для точки $(6,6,5,2)$, которая обозначена на рис. 1.2 крестиком, ближайшей точкой выборки будет точка $(6,4,5,6)$, которая принадлежит третьему классу ($y = 3$), поэтому точке $(6,6,5,2)$ также припишем третий класс, т.е. $f(6,6,5,2) = 3$.

Введём формальное обозначение для выборки

$$S = ((x_1^i, x_2^i, y^i) | i = 1, \dots, N) = ((x^i, y^i) | i = 1, \dots, N),$$

где $x^i = (x_1^i, x_2^i)$, N – количество объектов (объём выборки).

Здесь мы просто ввели буквенные обозначения для таблицы данных и её содержимого.

Теперь решающую функцию, получаемую методом ближайшего соседа, можно записать формально

$$f(x) = y^{i_x}, \quad i_x = \arg \min_i \rho(x, x^i),$$

где $\arg \min$ – аргумент минимума, т.е. в данном случае это индекс i , при котором достигается минимум расстояния.

Существует модификация метода, когда вместо одной точки находятся несколько ближайших точек выборки и приписывается класс, к которому принадлежит большинство из них. Это метод k ближайших соседей (или kNN).

1.2 Метод ближайшего соседа для вещественной целевой переменной

В предыдущем разделе переменная Y принимала конечный набор значений, на которых не был определён порядок (так как на сортах ирисов не определено отношение «больше-меньше»). В таких случаях задача построения решающей функции называется задачей распознавания образов или задачей классификации (с «учителем»).

Если целевая переменная Y является вещественной, то говорят о задаче восстановления зависимостей. Задачу построения решающей функции при вещественной Y можно также называть задачей регрессионного анализа, хотя это будет не совсем точно, поскольку регрессионный анализ предполагает, что мы оцениваем функцию регрессии, т.е. функцию условного математического ожидания. Однако, функция регрессии является оптимальным решением, если в качестве критерия используется средний квадрат отклонения (MSE). Если, например, критерием является средний модуль отклонения (MAE), то оптимальным решением будет не регрессия, а условная медиана.

Кроме того, классическая постановка регрессионного анализа подразумевает, что значения переменных, кроме целевой, не являются случайными. Мы же полагаем все переменные случайными величинами.

Однако, термин «восстановление зависимостей» тоже не является точным, поскольку подразумевает, что есть некоторая «истинная» зависимость, которую нужно восстановить. Вместе с тем, в реальных задачах чаще всего никакой «объективной» зависимости нет, есть только статистическая (вероятностная) связь (зависимость) между переменными. А функциональную зависимость (решающую функцию) мы не восстанавливаем, а конструируем.

Поэтому будем пользоваться термином «построение решающей функции».

В качестве примера задачи построения решающей функции для вещественной (непрерывной) целевой переменной рассмотрим задачу «Wine Quality» из репозитория UCI.

База данных для этой задачи описывает несколько тысяч образцов вин, которые характеризуются одиннадцатью переменными, соответствующими измерениям различных химических показателей. Целевая переменная Y принимает целые значения от 3 до 8 и является экспертной оценкой качества вина.

Поскольку дробные оценки качества не противоречат содержательному смыслу задачи, будем считать переменную Y вещественной.

Как и в предыдущем случае, уменьшим размерность задачи. Выберем две переменных:

X_1 – концентрация летучих кислот (volatile acidity),

X_2 – концентрация лимонной кислоты (citric acid).

Из всего множества объектов выберем по пять образцов для каждого значения качества. Заметим, что после такого сокращения данных решение, которое будет получено, нельзя переносить на исходную задачу, фактически мы сформулировали новую учебную задачу на основе данных UCI.

Сформированная выборка приведена в таблице на рис. 1.3 и визуализирована на рис. 1.4, где цифры рядом с точками отражают значения целевой переменной.

Задача заключается в построении решающей функции, которая по значениям химических показателей позволяет оценить качество вина.

Пусть, например, мы получили образец, для которого $x_1 = 0,5$, $x_2 = 0,35$. Он изображён точкой A на рис. 1.4. Что можно сказать о его качестве? Наиболее похожим из выборки является объект $i = 2$ со значениями $(0,44, 0,42)$, для которого $y = 3$. Таким образом, по методу ближайшего соседа оценка качества для объекта A также составит 3.

i	x_1^i	x_2^i	y^i	i	x_1^i	x_2^i	y^i	i	x_1^i	x_2^i	y^i
1	1,02	0,02	3	11	0,7	0,13	5	21	0,36	0,34	7
2	0,44	0,42	3	12	0,59	0,01	5	22	0,3	0,41	7
3	0,61	0,49	3	13	0,49	0,49	5	23	0,59	0	7
4	0,76	0,02	3	14	0,82	0,29	5	24	0,34	0,4	7
5	1,19	0	3	15	0,69	0,49	5	25	0,59	0,06	7
6	0,33	0,32	4	16	0,58	0,56	6	26	0,35	0,53	8
7	0,58	0	4	17	0,56	0,24	6	27	0,62	0,67	8
8	0,92	0,27	4	18	0,39	0,47	6	28	0,49	0,03	8
9	0,48	0,2	4	19	0,49	0,1	6	29	0,32	0,45	8
10	1,13	0,09	4	20	0,51	0,13	6	30	0,3	0,56	8

Рис. 1.3: Данные по задаче оценивания качества вин.

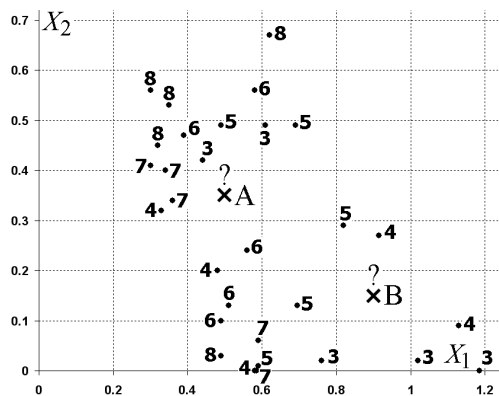


Рис. 1.4: Визуализация данных из таблицы 1.3.

Однако заметим, что недалеко от A находятся и точки, для которых $y = 7$. И при незначительном смещении точки A оценка по методу ближайшего соседа может сильно измениться.

Подобное скачкообразное изменение решающей функции представляется неоправданным, поскольку разумно считать, что качество продукта в данном случае является непрерывной функцией характеристик.

Для построения гладкой решающей функции применим так называемый ядровой (kernel-based) метод, также известный как метод потенциальных функций.

В этом методе решающая функция строится в виде

$$f(x) = \frac{1}{w} \sum_{i=1}^N y^i w_i, \quad w = \sum_{i=1}^N w_i,$$

где $w_i = \varphi(\rho(x, x^i))$, $\rho(x, x^i)$ – расстояние между заданной точкой и объектом выборки, а $\varphi(\cdot)$ – потенциальная или ядровая функция.

В качестве потенциальной функции можно использовать любую монотонно убывающую функцию, например, ядро Коши

$$\varphi(z) = \frac{1}{1 + \left(\frac{z}{r}\right)^2},$$

где r – параметр, задающий скорость убывания функции.

На содержательном уровне идея метода в том, что решение в каждой точке есть взвешенное среднее по выборочным значениям, причём, чем ближе выборочная точка к данной, тем в большем весе она учитывается.

Для примера вычислим оценки качества в точках A и B . При $r = 0,1$ в точке A имеем $f(0,5, 0,35) \approx 5,49$, для точки B получаем $f(0,9, 0,15) \approx 4,7$.

Данный результат выглядит достаточно разумным. Действительно, согласно выборке, качество вина выше при меньшей концентрации летучих кислот (в т.ч. уксусной), и большей концентрации лимонной кислоты.

1.3 Характеристика метода прецедентов

1.3.1 Варианты метода прецедентов

Можно выделить следующие основные разновидности метода прецедентов:

- k -ближайших соседей (прогноз на основе объектов, ближайших к исследуемому),
- парzenовское окно (прогноз на основе объектов, попавших в окрестность исследуемого объекта),
- ядровое сглаживание (вклад объекта в прогноз есть некоторая функция расстояния).

В методе парзеновского окна фиксируется не число соседей, а форма и размер «окна», т.е. окрестности, из которой берутся соседи.

Сглаживание с использованием ядер есть наиболее общий вариант метода прецедентов. При этом функция ядра может быть как функция непосредственно расстояния, так и функция ранга по степени близости (т.е. функция номера «соседа» в порядке близости). Тогда методы kNN и парзеновского окна соответствуют ядру в виде пороговой функции.

1.3.2 Свойства метода прецедентов

Метод прецедентов, несмотря на естественность и очевидность, на практике работает относительно плохо, хотя есть исключения.

Недостатки метода:

- равное использование всех переменных, неустойчивость к «шумовым переменным», особенно при большой размерности,
- неиспользование возможной независимости переменных,
- неиспользование априорных знаний.

Основным недостатком метода является его малая пригодность для пространств большой размерности. При большом числе переменных понятие близости между точками становится малосодержательным и не отражает интуицию.

Представим, что в некоторой задаче есть переменная, по которой два класса безошибочно разделяются. И есть ещё 99 переменных, на которых распределения обоих классов почти совпадают (такие переменные называют «шумовыми»). В такой задаче kNN даст очень низкую точность, поскольку на расстояние между объектами влияют все переменные, и объекты, близкие по информативной переменной, весьма вероятно будут далёкими во всём пространстве.

Существуют методы, существенно более эффективные (на большинстве практических задач).

Тем не менее, метод kNN вполне можно использовать в некоторых случаях.

Во-первых, это задачи небольшой размерности, где расстояние «работает».

Во-вторых, существуют методы отбора информативных переменных, которые иногда позволяют уменьшить размерность задачи.

В-третьих, метод kNN иногда используют на случайных подпространствах небольшой размерности, чтобы сформировать новые признаки в методе стэкинга (позже его обсудим).

И в-четвёртых, существует важный подход, называемый *metric learning*. В этом подходе нейросеть обучается специально таким образом, чтобы сформировать признаковое описание (*embedding*) объекта именно так, чтобы объекты одного класса (или близкие по целевой переменной) были близкими в этом пространстве. Фактически само пространство строится именно так, чтобы kNN работал, и он в этом случае работает.

1.4 Примеры кода

Для наглядности здесь приведены примеры кода, позволяющего визуализировать данные, упомянутые в главе.


```

# Загружаем библиотеки:
import numpy as np
import pandas as pd
from sklearn import datasets
import seaborn as sns

# Загружаем набор данных Ирисы:
iris = datasets.load_iris()

# Формируем таблицу:
df = pd.DataFrame(iris['data'],
                  columns = iris['feature_names'])

# Добавляем столбец с целевой переменной:
df['target'] = iris['target']

# Заменяем номера классов названиями сортов:
df['target'] = df['target'].map(lambda x :
                               iris['target_names'][x])

# Смотрим, что получилось:
df

```

Результат выполнения кода приведён на рис. 1.5. Здесь видим полную таблицу данных Iris. Дополнительно показан результат операции вычисления моментов по классам.

Код запускался в среде jupyter-notebook, которая предоставляет запускать код частями и видеть результат выполнения каждого фрагмента. Также важной особенностью является то, что после модификации кода нет необходимости запускать программу с начала — можно продолжить выполнение в текущем контексте.

Пожалуй, для первоначального знакомства с языком python данная среда подходит лучше всего.

На рис. 1.6 приведён результат выполнения кода

```
sns.pairplot(df, hue = 'target')
```

Показан только фрагмент рисунка, который в полном виде представляет собой 4×4 таблицу картинок и имеет слишком большой размер.

Видим, что библиотека seaborn позволяет одним вызовом получать достаточно сложные и информативные иллюстрации.

df

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

df.groupby(['target']).agg(['mean', 'std'])

	sepal length (cm)		sepal width (cm)		petal length (cm)		petal width (cm)	
target	mean	std	mean	std	mean	std	mean	std
setosa	5.006	0.352490	3.428	0.379064	1.462	0.173664	0.246	0.105386
versicolor	5.936	0.516171	2.770	0.313798	4.260	0.469911	1.326	0.197753
virginica	6.588	0.635880	2.974	0.322497	5.552	0.551895	2.026	0.274650

Рис. 1.5: Выполнение кода в jupyter-notebook.

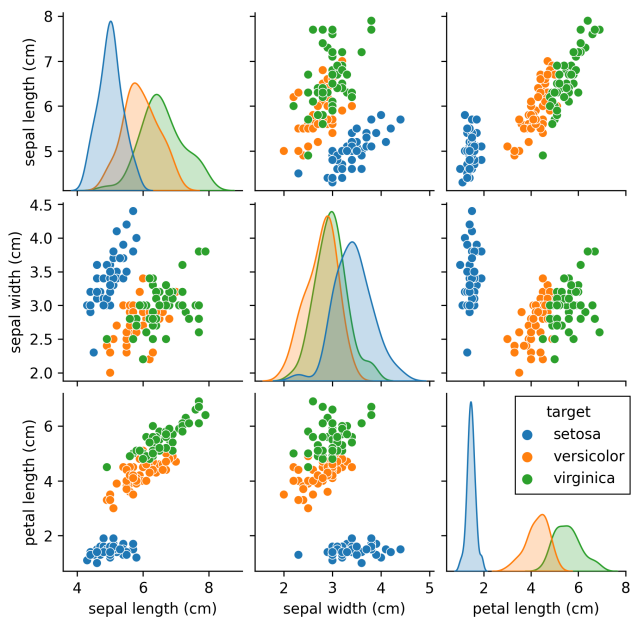


Рис. 1.6: Визуализация средствами seaborn.

В данном пособии примеров кода будет относительно мало, поскольку большой объём кода был бы неуместен. При этом примеры отобраны только наиболее важные с точки зрения иллюстрации работы методов или демонстрации важных конструкций языка.

Автор настоятельно рекомендует запустить все предложенные примеры и понять работу кода, в том числе внося в него изменения и наблюдая результат.

Глава 2

(Квази)линейные методы классификации

В данной главе рассмотрены методы классификации, в которых решающая функция является линейной, либо становится линейной в подходящем пространстве переменных.

Список методов:

- линейный и квадратичный дискриминант,
- дискриминант Фишера,
- логистическая регрессия,
- наивный байесовский классификатор,
- машина опорных векторов.

2.1 Решающая функция для нормальных распределений

Нормальное распределение является, пожалуй, наиболее часто используемым в теории вероятностей и математической статистике. В задачах анализа данных зачастую нет оснований предполагать нормальность распределений, однако, в ряде случаев такую модель оправдано использовать.

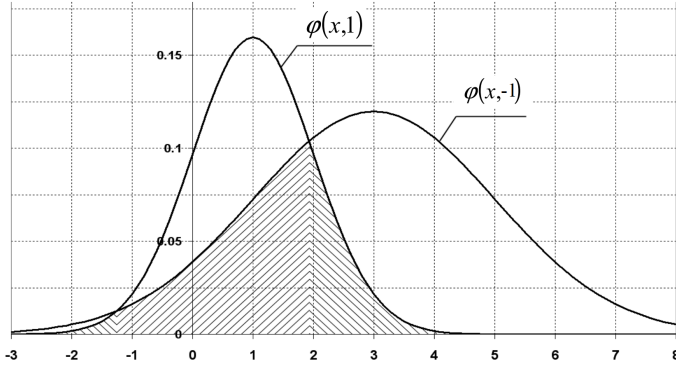


Рис. 2.1: Совместная плотность вероятности.

Рассмотрим, как выглядит оптимальное решение, если распределения известны.

2.1.1 Одномерный случай

Для простоты будем рассматривать случай двух классов, т.е. $Y = \{-1, 1\}$, и пусть имеется одна непрерывная переменная X .

Предположим, что заданы распределение $P(y) = P_y$ и условные плотности вероятности $\varphi_y(x)$, $y \in \{-1, 1\}$, причём плотности являются нормальными

$$\varphi_y(x) = \frac{1}{\sqrt{2\pi}\sigma_y} e^{-\frac{(x-\mu_y)^2}{2(\sigma_y)^2}}.$$

Через заданные величины можно выразить совместную плотность вероятности

$$\varphi(x, y) = \varphi_y(x)P(y).$$

В качестве примера на рис. 2.1 приведён график совместной плотности при $P(1) = 0,4$, $P(-1) = 0,6$, $\mu_1 = 1$, $\sigma_1 = 1$, $\mu_{-1} = 3$, $\sigma_{-1} = 2$.

Оптимальная (или байесовская) решающая функция будет выглядеть следующим образом

$$f^*(x) = \arg \max_y \varphi(x, y). \quad (2.1)$$

При $\varphi(x, 1) = \varphi(x, -1)$ функция $\arg \max$ возвращает оба значения, т.е. множество $\{-1, 1\}$. В этом случае значение байесовской решающей функции может выбираться произвольно, как 1 так и -1 , причём при практической реализации часто имеет смысл делать случайный (равновероятный) выбор между этими значениями.

Заметим, что в определении байесовской решающей функции 2.1 формула Байеса не использовалась, поэтому термин «байесовская», пожалуй, является не вполне удачным, однако, он устоявшийся, и мы будем его использовать.

На рис. 2.1 компоненты совместной плотности имеют две точки пересечения: при $x = a \approx -1,26$ и при $x = b \approx 1,93$. Оптимальная решающая функция

$$f^*(x) = \begin{cases} 1, & x \in [a, b] \\ -1, & x \notin [a, b] \end{cases}.$$

Вероятность ошибочной классификации для оптимального решения есть

$$R^* = P(f^*(x)) = \int_{f^*(x)=-1} \varphi(x, 1) dx + \int_{f^*(x)=1} \varphi(x, -1) dx.$$

Для приведённого примера

$$\begin{aligned} R^* &= \int_{-\infty}^a \varphi(x, 1) dx + \int_a^b \varphi(x, -1) dx + \int_b^{\infty} \varphi(x, 1) dx = \\ &= \frac{1}{2}P(1) \left(1 + \Phi \left(\frac{a - \mu_1}{\sigma_1} \right) + 1 - \Phi \left(\frac{b - \mu_1}{\sigma_1} \right) \right) + \\ &+ \frac{1}{2}P(-1) \left(\Phi \left(\frac{b - \mu_{-1}}{\sigma_{-1}} \right) - \Phi \left(\frac{a - \mu_{-1}}{\sigma_{-1}} \right) \right) \approx 0,24. \end{aligned}$$

Полученное значение есть площадь заштрихованной области на рис. 2.1.

Здесь интегралы от нормальной плотности выражены через так называемую функцию Лапласа

$$\Phi(z) = \frac{2}{\sqrt{2\pi}} \int_0^z e^{-\frac{t^2}{2}} dt,$$

значения которой могут быть найдены в справочных таблицах либо вычислены на компьютере.

Введём функции условной вероятности классов

$$g_1(x) = P(y = 1 | x) = \varphi_1(x) \frac{P_1}{\varphi(x)}, \quad g_{-1}(x) = 1 - g_1(x),$$

где

$$\varphi(x) = \varphi_1(x)P_1 + \varphi_{-1}(x)P_{-1}.$$

Здесь использована формула Байеса.

Байесовская решающая функция может быть записана как

$$f^*(x) = \arg \max_y g_y(x).$$

Если максимум достигается на обоих классах, можем выбрать любой, например, класс 1.

Поскольку функции $g_1(x)$ и $g_{-1}(x)$ в рассматриваемом случае положительны, их сравнение друг с другом эквивалентно сравнению их отношения с единицей, что, в свою очередь, эквивалентно сравнению логарифма отношения с нулём, поэтому байесовскую решающую функцию можно представить в виде

$$f^*(x) = \begin{cases} 1, & l(x) \geq 0 \\ -1, & l(x) < 0 \end{cases},$$

где

$$l(x) = \ln \frac{g_1(x)}{g_{-1}(x)} = \ln \varphi_1(x) - \ln \varphi_{-1}(x) + \ln \frac{P(1)}{P(-1)}.$$

Функцию $l(x)$ будем называть разделяющей функцией.

Мы также можем использовать стандартную функцию знака (сигнум)

$$\text{sign}(x) = \begin{cases} 1, & x > 0 \\ 0, & x = 0 \\ -1, & x < 0 \end{cases}.$$

Можем записать $f^*(x) = \text{sign}(l(x))$. В этом случае мы допускаем, что на границе классов прогноз будет 0, что можно интерпретировать как «отказ от прогноза». Это ни на что не влияет, поскольку происходит на множестве меры 0.

Подставляя нормальные плотности, получаем

$$l(x) = \frac{(x - \mu_{-1})^2}{(\sigma_{-1})^2} - \frac{(x - \mu_1)^2}{(\sigma_1)^2} + \ln \frac{\sigma_{-1}}{\sigma_1} + \ln \frac{P(1)}{P(-1)}.$$

Видим, что $l(x) = 0$ представляет собой квадратное уравнение, которое имеет либо два корня, либо один корень — в случае $\sigma_1 = \sigma_{-1}$, либо не имеет корней — если одна из компонент совместной плотности целиком лежит под другой (также возможен очевидный вырожденный случай бесконечного множества корней — при полном совпадении компонент).

Заметим, что имеет место следующее соотношение

$$g_1(x) = P(y = 1|x) = \frac{1}{1 + e^{-l(x)}} = \sigma(l(x)).$$

Здесь $\sigma(z) = \frac{1}{1+e^{-z}}$ — так называемая логистическая функция (иногда также называемая сигмоидом или логит-функцией). Данная функция, как мы увидим в дальнейшем, очень часто возникает при решении задачи классификации и играет важную роль.

2.1.2 Многомерный случай

Рассмотрим теперь случай нескольких переменных $X_1, \dots, X_n$.

Условные плотности вероятности $\varphi_y(x)$ предполагаются нормальными и задаются параметрами μ_y и λ_y , где μ_y — вектор средних (математических ожиданий) для класса y , а λ_y — ковариационная матрица.

Оптимальная решающая функция записывается, так же как и в одномерном случае, через разделяющую функцию

$$l(x) = \ln \varphi_1(x) - \ln \varphi_{-1}(x) + \ln \frac{P(1)}{P(-1)}.$$

Решение уравнения $l(x) = 0$ задает границу между классами и называется разделяющей поверхностью.

Подставим в разделяющую функцию аналитические выражения для нормальных плотностей

$$\varphi_y(x) = \frac{1}{(2\pi)^{n/2} |\lambda_y|^{n/2}} e^{-\frac{1}{2} Q_y(x)},$$

где $Q_y(x) = (x - \mu_y)'(\lambda_y)^{-1}(x - \mu_y)$.

Штрих в записи квадратичной формы обозначает транспонирование вектора.

После преобразований имеем

$$2l(x) = Q_{-1}(x) - Q_1(x) + \ln |\lambda_{-1}| - \ln |\lambda_1| + 2 \ln P(1) - 2 \ln P(-1).$$

Далее преобразуем

$$\begin{aligned} Q_{-1}(x) - Q_1(x) = & (x - \mu_{-1})'(\lambda_{-1})^{-1}(x - \mu_{-1}) - (x - \mu_1)'(\lambda_1)^{-1}(x - \mu_1) = \\ & x'((\lambda_{-1})^{-1} - (\lambda_1)^{-1})x - (\mu_{-1}'(\lambda_{-1})^{-1} - \mu_1'(\lambda_1)^{-1})x - \\ & x'((\lambda_{-1})^{-1}\mu_{-1} - (\lambda_1)^{-1}\mu_1) + \\ & \mu_{-1}'(\lambda_{-1})^{-1}\mu_{-1} - \mu_1'(\lambda_1)^{-1}\mu_1. \end{aligned}$$

Наконец получаем

$$2l(x) = x'Ax + bx + c, \quad (2.2)$$

где

$$\begin{aligned} A &= (\lambda_{-1})^{-1} - (\lambda_1)^{-1}, \\ b &= 2\mu_1(\lambda_{-1})^{-1} - 2\mu_{-1}(\lambda_1)^{-1}, \\ c &= \mu_{-1}'(\lambda_{-1})^{-1}\mu_{-1} - \mu_1'(\lambda_1)^{-1}\mu_1 \\ &\quad + \ln \frac{|\lambda_{-1}|}{|\lambda_1|} + 2 \ln \frac{P_1}{P_{-1}}. \end{aligned}$$

При получении выражения для b использована коммутативность скалярного произведения векторов и коммутативность произведения вектора на симметричную матрицу (напомним, что ковариационная матрица симметрична).

Уравнение $l(x) = 0$ в общем случае задает поверхность второго порядка, которая при $\lambda_1 = \lambda_{-1}$, то есть при равенстве ковариационных матриц, вырождается в гиперплоскость.

Заметим, что ковариационные матрицы являются положительно определенными, ввиду чего линии (поверхности) уровня нормального распределения представляют собой эллипсоиды. При

этом разделяющая поверхность может оказаться любой поверхностью второго порядка: эллипсоидом, гиперboloидом, параболоидом, конусом, цилиндром.

2.1.3 Оценивание параметров

Пусть имеется выборка, представленная прямоугольной матрицей

$$S = ((x^i, y^i) | i = 1, \dots, N), \quad x^i = (x_1^i, \dots, x_n^i).$$

Параметр N представляет собой число независимых реализаций системы случайных величин и называется объемом выборки, n – число переменных.

Сформируем множество индексов (номеров) объектов, принадлежащих первому классу: $I_1 = \{i | y^i = 1\}$. Аналогично, $I_{-1} = \{i | y^i = -1\}$. Тогда $N_y = |I_y|$ есть число объектов класса y в выборке, $N_1 + N_{-1} = N$.

Проще всего оценить априорные (безусловные) вероятности классов, а именно: $\tilde{P}_y = \frac{N_y}{N}$, где $\tilde{P}_y$ – оценка для P_y .

Обозначим через $\tilde{\mu}_y$ оценку вектора μ_y .

Компоненты вектора средних и ковариационной матрицы для каждого класса оцениваются по формулам

$$\begin{aligned} \tilde{\mu}_y &= \frac{1}{N_y} \sum_{i \in I_y} x^i, \\ \tilde{\lambda}_y &= \frac{1}{N_y} \sum_{i \in I_y} (x^i - \tilde{\mu}_y)(x^i - \tilde{\mu}_y)'. \end{aligned} \quad (2.3)$$

где I_y – множество индексов объектов класса y из выборки.

Здесь произведение векторов вычисляется по правилу умножение матриц, причём (поскольку транспонирован правый сомножитель) умножается вектор-столбец на вектор-строку, в результате чего получается квадратная матрица оценок ковариаций.

Оценки 2.3 являются оценками максимального правдоподобия. Можно использовать несмещённую версию оценок, но на типичных размерах выборки различие пренебрежимо.

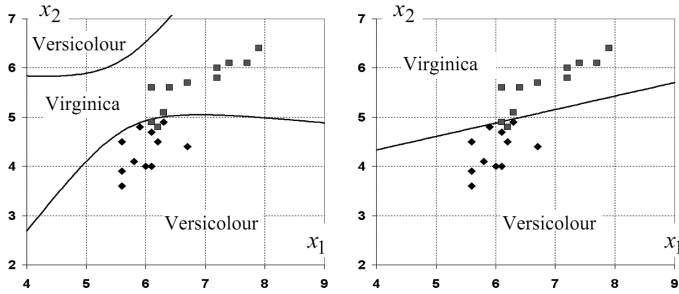


Рис. 2.2: Квадратичный и линейный дискриминант в задаче Iris.

Для построения выборочной разделяющей поверхности достаточно теперь в уравнение подставить вместо параметров распределений найденные оценки.

Случай равных матриц ковариации.

Если известно, что распределения обоих классов имеют одинаковые матрицы ковариации, то нужно всю выборку использовать для вычисления оценки $\tilde{\lambda}$ матрицы $\lambda = \lambda_1 = \lambda_{-1}$. Оценка находится следующим образом

$$\tilde{\lambda} = \frac{1}{N} \sum_{y \in \{1, -1\}} \sum_{i \in I_y} (x^i - \tilde{\mu}_y)(x^i - \tilde{\mu}_y)'. \quad (2.4)$$

Остальные параметры оцениваются так же, как в общем случае.

В качестве примера рассмотрим задачу об ирисах, изложенную в главе 2.

Поскольку мы разобрали метод применительно к случаю двух классов, выберем для классификации два вида ирисов: Iris Versicolour и Iris Virginica.

Если предположить ковариационные матрицы классов различными, то, оценив параметры распределения, получим разделяющую кривую, которая оказалась гиперболой, график которой приведён на левой диаграмме рис. 2.2.

Если предположить равенство ковариационных матриц, то получим разделяющую прямую, график которой приведён на правой диаграмме рис. 2.2.

На практике обычно нет информации о том, являются ли ковариационные матрицы классов одинаковыми, поэтому решение о способе их оценивания приходится принимать эмпирически.

Можно, например, применить статистический критерий проверки гипотезы о равенстве ковариационных матриц. Но на практике это нецелесообразно.

Раумный вариант: при большом объеме выборки оценивать матрицы для каждого класса отдельно, при малых выборках — оценивать одну матрицу и строить линейную разделяющую поверхность.

Дело в том, что квадратичный дискриминант требует оценить порядка n^2 параметров (напомним, что n — число переменных), а линейный дискриминант — порядка n параметров. Может показаться, что линейный дискриминант тоже оценивает ковариационную матрицу (где n^2 значений), но на самом деле важно именно число степеней свободы в решении, а линейное решение определяется уравнением разделяющей гиперплоскости, т.е. имеет $n + 1$ параметров.

Здесь мы впервые сталкиваемся с эффектом, что при малых выборках следует строить более простое решающее правило, даже если это упрощение вносит заведомую погрешность. В дальнейшем такая ситуация будет частой.

Решение о выборе между линейным и квадратичным дискриминантом целесообразно принимать на основе кросс-валидации.

Таким образом, даже казалось бы в простейшем случае, когда форма распределений известна, а именно нормальна, оптимальный метод построения решающей функции по конечной выборке неизвестен. При «малых» выборках мы строим линейный дискриминант (даже если точно знаем, что ковариационные матрицы не равны), при «больших» — квадратичный. Кажется, что в подобной ситуации должны быть и промежуточные варианты.

2.1.4 Фильтрация нетипичных значений

Описанный метод сильно чувствителен к «выбросам», т.е. нетипичным значениям переменных, сильно отличающимся от нормальных. Такие значения могут быть вызваны как ошибками из-

мерений (или ошибками переноса значений в таблицу), так и наличием нехарактерных объектов в выборке (например, растение повреждено вредителями и не выросло до обычных размеров).

Для решения проблемы выбросов существуют отработанные методы робастных оценок. Общая идея таких оценок заключается в следующем. Сначала строятся оценки параметров по всей выборке. После чего вся выборка упорядочивается по значению правдоподобия, рассчитанного по этим параметрам. Далее некоторая доля (например 1%) наименее правдоподобных точек выборки отбрасывается, после чего параметры вновь оцениваются по оставшимся объектам. Эту процедуру можно повторять несколько раз.

Такой алгоритм является достаточно универсальным (может применяться с разными методами) и позволяет эффективно отфильтровывать нетипичные значения в данных. Следует только иметь в виду, что после такой фильтрации в оценки условных вероятностей классов нужно делать поправки. Грубо говоря, если мы удалили из выборки 1% объектов, то ни в одной точке пространства мы не должны давать для функции $g_y(x)$ оценку меньше 0,01 и больше 0,99.

2.1.5 Свойства метода

Можно отметить следующие особенности рассмотренного метода:

- основан на предположении нормальности распределений,
- чувствителен к «выбросам», но эта проблема решается,
- имеет низкую вычислительную трудоёмкость, решение в аналитическом виде,
- выбор между линейным и квадратичным решениями не однозначен.

Первую особенность как правило позиционируют как недостаток метода. Однако её можно считать, наоборот, достоинством. Аргументация этого следующая.

Если в какой-то задаче мы знаем, что распределения классов нормальны, это, очевидно, является некоторым обоснованием применения метода (линейного или квадратичного дискриминанта).

Если же мы не имеем оснований полагать распределения нормальными, или даже точно знаем, что они не являются нормальными, то из этого совершенно не следует, что данный метод в такой задаче работать не будет.

Это совершенно общее соображение: если что-то работает при некоторых предположениях, то это не значит, что оно не работает при нарушении предположений. Достаточность условий не подразумевает их необходимость.

Если распределения не являются нормальными, мы всё равно можем применять линейный и квадратичный дискриминант, но уже как чисто эвристический метод.

Заметим, что почти все (другие) методы машинного обучения изначально являются эвристическими. Это не значит, что они не имеют обоснования. Про обоснованность методов машинного обучения будем говорить в седьмой главе.

2.2 Дискриминант Фишера

2.2.1 Идея метода

Идея метода заключается в выборе направления, при проецировании выборки на которое образы классов оказываются наиболее удалёнными друг от друга.

Это выражается в максимизации критерия

$$\Phi(w) = \frac{(\tilde{\mu}_{w,1} - \tilde{\mu}_{w,-1})^2}{\tilde{S}_{w,1} + \tilde{S}_{w,-1}}, \quad (2.5)$$

где $\tilde{\mu}_{w,y} = \frac{1}{N_y} \sum_{i \in I_y} wx_i$ — среднее,

а $\tilde{S}_{w,y} = \frac{1}{N_y} \sum_{i \in I_y} (wx_i - \tilde{\mu}_{w,y})^2$ — средний квадрат отклонений проекций.

Этот критерий имеет очень простой содержательный смысл: два множества точек тем дальше друг от друга, чем дальше их

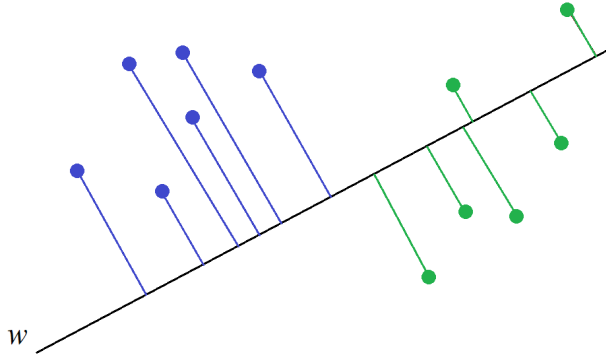


Рис. 2.3: Пример проецирования выборки на направление.

средние и чем меньше разброс точек.

Требуется найти оптимальное направление.

Оказывается, что в 2.5 максимум $\Phi(w)$ достигается при

$$w_{\Phi} = \tilde{S}^{-1}(\tilde{\mu}_1 - \tilde{\mu}_{-1}), \quad (2.6)$$

где $\tilde{\mu}_y$ – среднее точек выборки y -го класса, а $\tilde{S}_y$ – выборочная ковариационная матрица y -го класса, $\tilde{S} = \tilde{S}_1 + \tilde{S}_{-1}$.

Полученное выражение неожиданно оказывается практически совпадающим с 2.2 при равных матрицах ковариаций.

Отличия незначительны, но их стоит отметить.

Дискриминант Фишера не предполагает равенства ковариационных матриц и использует усреднённую матрицу по классам. В отличие от 2.4, классы здесь дают вклад с равными весами, независимо от соотношения классов в выборке.

На самом деле это свойство можно изменить, скорректировав критерий 2.5, но это бы противоречило идее критерия. Здесь мы не предполагаем равенство ковариационных матриц, поэтому оцениваем каждую отдельно, а потом усредняем (с равным весом). А в линейном дискриминанте предполагаем равенство матриц, поэтому оцениваем одну матрицу по полной выборке.

Другое отличие в том, что дискриминант Фишера даёт только направление, но не описывает способ постановки границы между проекциями классов. Предполагается, что если задача сведена к одномерной, то разумным образом разделить классы не составит труда (например, непосредственной минимизацией числа ошибок).

2.2.2 Свойства метода

Замечания.

- Метод не требует никаких вероятностных предположений.
- Выражение для w_f очень похоже на выражение для нормали к разделяющей гиперплоскости для случая нормальных распределений с равными матрицами ковариаций.
- Метод предполагает оценивание по выборке только $n + 1$ параметров и не требователен к объёму выборки.
- Метод неустойчив к «выбросам», но позволяет использовать приём робастного оценивания.

Мы получили нетривиальный факт, что метод, выведенный из очень сильных предположений (нормальность и равенство ковариаций) можно получить вообще без всяких предположений, только лишь на основе геометрических эвристик.

Это позволяет сделать вывод, что метод, выведенный при сильных предположениях, может оставаться пригодным и при нарушении этих предположений.

2.3 Логистическая регрессия

2.3.1 Идея метода

Рассмотрим функцию условной вероятности для класса 1

$$g(x) = P(y = 1|x) = \frac{P(1)\varphi_1(x)}{P(1)\varphi_1(x) + P(-1)\varphi_{-1}(x)} = \frac{1}{1 + e^{-l(x)}},$$

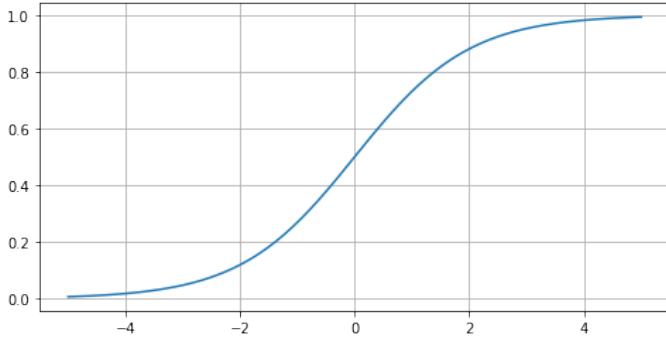


Рис. 2.4: График функции $\sigma(z)$.

где $l(x)$ – разделяющая функция.

Подставив нормальные плотности при равных матрицах ковариаций, получим

$$g(x) = \frac{1}{1 + e^{-(wx+w_0)}} = \sigma(wx + w_0),$$

где w и w_0 есть переобозначение параметров b и c соответственно из линейного дискриминанта 2.2.

Здесь $\frac{1}{1+e^{-z}}$ – так называемая логистическая функция (иногда также называемая сигмоидом или логит-функцией).

Определить (полную) вероятностную модель данных мы можем либо задав совместную плотность $\varphi(x, y)$, либо задав пару $\varphi_y(x)$, $P(y)$, либо задав пару $P(y|x)$, $\varphi(x)$. Все способы эквивалентны. Два последних варианта переводятся друг в друга формулой Байеса.

В методе квадратичного дискриминанта мы предполагали нормальность $\varphi_y(x)$ и затем находили $g_y(x) = P(y|x)$.

Возникает разумный вопрос: зачем делать предположения о форме $\varphi_y(x)$, если нам нужна $g_y(x)$. Почему не сделать сразу предположение о форме $g_y(x)$?

Оказывается, что так сделать можно и именно в этом заключается идея логистической регрессии.

При этом форма безусловной плотности $\varphi(x)$ нас не интересует, и мы о ней делать предположений не будем. В этом смысле

мы будем строить неполную вероятностную модель. Эта модель ничего не говорит о вероятности появления (правдоподобии) x , но описывает условную вероятность $P(y | x)$.

Параметры такой модели оказывается возможным оценивать уже привычным методом максимального правдоподобия. Мы подбираем такие параметры, чтобы вероятность имеющейся выборки была максимальна.

2.3.2 Оценивание параметров

Метод логистической регрессии основан на оценивании функции условной вероятности моделью $\tilde{g}(x) = \sigma(\tilde{w}x + \tilde{w}_0)$, в которой $\tilde{w}$ и $\tilde{w}_0$ – настраиваемые параметры.

На практике параметры модели обычно оцениваются путём максимизации критерия правдоподобия (условной вероятности выборки).

$$-K_S(\tilde{w}, \tilde{w}_0) = \sum_{i \in I_1} \ln \tilde{g}(x^i) + \sum_{i \in I_{-1}} \ln(1 - \tilde{g}(x^i)).$$

В случае линейно разделимой выборки максимум правдоподобия достигается на ступенчатой функции, которая является пределом сигмоида, когда $w^2 \rightarrow \infty$.

Формально в классе сигмоидов максимум не достигается, поэтому алгоритм численной оптимизации не сойдётся.

Чтобы обеспечить как существование решения, так и его единственность, требуется регуляризация. Это некоторая добавка к критерию, обеспечивающая устойчивость решения.

В данной задаче такой добавкой может быть $\varkappa(w^2)$ – некоторая монотонная функция от нормы вектора направления.

2.3.3 Функция потерь

Мы впервые встретились с методом, который сводится к минимизации аддитивной функции выборки. Дальнейшие методы именно так и будут устроены.

Метод минимизирует функцию

$$K_S(\tilde{w}, \tilde{w}_0) = \varkappa(w^2) + \sum_{i=1}^N L(\sigma(\tilde{w}x^i + \tilde{w}_0), y^i), \quad (2.7)$$

где

$$L(p, y) = \begin{cases} -\ln(p), & y=1 \\ -\ln(1-p), & y=-1 \end{cases} \quad (2.8)$$

представляет собой логарифмическую функцию потерь.

Учитывая, что $\ln \sigma(-z) = \ln(1 - \sigma(z))$, выражение 2.7 можно преобразовать к виду

$$K_S(\tilde{w}, \tilde{w}_0) = \varkappa(w^2) + \sum_{i=1}^N -\ln(\sigma(y^i \cdot (\tilde{w}x^i + \tilde{w}_0))). \quad (2.9)$$

Вид получившейся функции потерь приведён на рис. 2.7

2.3.4 Свойства метода

Замечания.

- Метод логистической регрессии похож на линейный дискриминант, но ослабляет вероятностные предположения.
- На практике (почти) никто вероятностные предположения не проверяет.
- По сравнению с линейным дискриминантом метод логистической регрессии более устойчив в «выбросам».
- Для повышения устойчивости требуется регуляризатор.

Можно было ожидать, что логистическая регрессия окажется похожа на линейный дискриминант. Но она окажется больше похожа на метод опорных векторов.

2.4 Наивный байесовский классификатор

«Наивный» байесовский классификатор основан на предположении условной независимости переменных. Данное предположение редко обосновано в реальных задачах, поэтому метод и получил такое название.

2.4.1 Случай двух классов

Из формулы Байеса можем записать

$$g(x) = P(y = 1|x) = \frac{P(dx, y = 1)}{P(dx, y = 1) + P(dx, y = -1)} = \frac{1}{1 + \frac{1-p}{p} \cdot \frac{P(dx|y=-1)}{P(dx|y=1)}},$$

где $p = P(y = 1)$.

Данное выражение можно преобразовать в виду

$$g(x) = \sigma \left(z_0 + \ln \frac{P(dx|y = 1)}{P(dx|y = -1)} \right),$$

где введено обозначение $z_0 = \ln p - \ln(1 - p) = \sigma^{-1}(p)$, а $\sigma(z) = \frac{1}{1+e^{-z}}$ — уже знакомая нам логистическая функция (сигмоид).

Выразим правую часть

$$\ln \frac{P(dx|y = 1)}{P(dx|y = -1)} = \sigma^{-1}(g(x)) - z_0. \quad (2.10)$$

Для ряда читателей, возможно, окажется непривычной используемая здесь нотация в духе производных Радона-Никодима (производная по мере). В этом случае можно при чтении везде мысленно заменить $P(dx...)$ на плотность вероятности $\varphi(x...)$. Если непонятно, как это сделать, можно посмотреть выкладки следующего подраздела, где как раз используются плотности. Там мы выведем формулу для нескольких классов, из которых в частном случае получается то, что выведем сейчас.

Смысл использования производных по мере исключительно в том, чтобы сделать выкладки универсальными, а именно пригодными для случаев, когда пространство переменных образуют случайные величины любых типов: дискретные, (абсолютно)

непрерывные, сингулярные. В противном случае для дискретных и непрерывных величин потребуются отдельные выражения.

Поскольку переменные разных типов — типичная ситуация в анализе данных, оказывается полезным изучить такие конструкции как производные по мере и интегралы Лебега и Лебега-Стилтьеса.

Независимость переменных означает, что совместное распределение раскладывается в произведение маргинальных распределений (т.е. одномерных распределений по каждой переменной).

Поэтому для (условно) независимых переменных имеем

$$\frac{P(dx|y=1)}{P(dx|y=-1)} = \prod_{j=1}^n \frac{P(dx_j|y=1)}{P(dx_j|y=-1)}.$$

После логарифмирования получим равносильное выражение

$$\ln \frac{P(dx|y=1)}{P(dx|y=-1)} = \sum_{j=1}^n \ln \frac{P(dx_j|y=1)}{P(dx_j|y=-1)}.$$

Выразим обе части этого равенства через $g(x)$ и $g_j(x)$.

Аналогично формуле 2.10 мы можем вывести выражение для функций $g_j(x_j) = P(y=1|x_j)$, а именно

$$\ln \frac{P(dx_j|y=-1)}{P(dx_j|y=1)} = \sigma^{-1}(g_j(x)) - z_0.$$

Собрав всё вместе, получаем

$$\sigma^{-1}(g(x)) - z_0 = \sum_{j=1}^n (\sigma^{-1}(g_j(x_j)) - z_0). \quad (2.11)$$

Если ввести новые переменные $z_j = \sigma^{-1}(g_j(x_j)) - z_0$, то в пространстве этих переменных решающая функция будет иметь вид логистической регрессии

$$g(x) = \sigma \left(w_0 + \sum_{j=1}^n w_j z_j \right), \quad (2.12)$$

при $w_0 = z_0$, $w_j \equiv 1$.

Предположение о независимости переменных на самом деле редко выполняется, поэтому метод, на нём основанный, действительно может считаться наивным. Однако различные варианты частичной независимости переменных вполне типичны для реальных задач. Мы можем «ослабить» гипотезу независимости, полагая w_j в 2.12 свободными параметрами (и подбирая их самым обычным методом логистической регрессии).

Такая замена переменных (x_j на z_j) называется target encoding (хорошего перевода нет).

Приём target encoding, т.е. кодирование значений категориальной переменной преобразованной прогнозируемой вероятностью, является одной из стандартных техник машинного обучения. Мы можем на переменных z_j использовать не только логистическую регрессию, но и вообще любой метод машинного обучения.

2.4.2 Случай нескольких классов

В большинстве случаев мы не рассматриваем специально случай нескольких классов, поскольку для большинства методов переход от двух классов ко многим осуществляется типовыми приёмами, которые мы рассмотрим в главе, специально посвящённой общим приёмам.

Однако для рассматриваемого метода рассмотрение нескольких классов приводит к важным формулам.

Итак, цель — выразить условную вероятность заданного класса при условии конкретных значений всех переменных через условные вероятности этого класса по каждой переменной, т.е. требуется выразить $P(y = \omega | x)$ через $P(y = \omega | x_j)$, $j = 1, \dots, n$.

Введём обозначения

$$u_j^\omega = \ln P(y = \omega | x_j) - \ln P(y = \omega).$$

Для простоты выкладок предположим, что все переменные X_j — абсолютно непрерывные случайные величины, т.е., что существует (многомерная) плотность вероятности $\varphi(x)$. Для произвольных случайных величин выкладки требуют привлечения производных по мере и читатель может провести их самостоятельно по аналогии с выкладками предыдущего раздела.

Из формулы Байеса можем записать

$$P(y = \omega|x) = \frac{\varphi_{\omega}(x)P(\omega)}{\varphi(x)},$$

где $\varphi_{\omega}(x)$ – условная плотность вероятности, $P(\omega)$ – вероятность класса ω .

По предположению о независимости переменных

$$\varphi_{\omega}(x) = \prod_{j=1}^n \varphi_{\omega,j}(x_j).$$

Ещё раз применим формулу Байеса:

$$\varphi_{\omega,j}(x_j) = \frac{P(y = \omega|x_j)\varphi_j(x_j)}{P(\omega)}.$$

Соберём всё воедино

$$P(y = \omega|x) = \frac{P(\omega)}{\varphi(x)} \prod_{j=1}^n \frac{P(y = \omega|x_j)\varphi_j(x_j)}{P(\omega)}.$$

Обозначим

$$C(x) = \frac{1}{\varphi(x)} \prod_{j=1}^n \varphi_j(x_j)$$

и

$$\zeta_{\omega}(x) = \ln P(\omega) + \sum_{j=1}^n (\ln P(y = \omega|x_j) - \ln P(\omega)) =$$

$$\ln P(\omega) + \sum_{j=1}^n u_j^{\omega}.$$

В этих обозначениях

$$P(y = \omega|x) = C(x)e^{\zeta_{\omega}(x)}.$$

Величина $C(x)$ нам неизвестна. К счастью, её можно найти из условия, что сумма вероятностей всех классов (при любом условии) равна 1. Получаем

$$P(y = \omega|x) = \frac{e^{\zeta_{\omega}(x)}}{\sum_{v=1}^k e^{\zeta_v(x)}}. \quad (2.13)$$

Цель достигнута.

Выражение, что мы получили, есть ни что иное как хорошо известная (например из нейросетей) функция softmax, которая является обобщением логистической функции.

Убедимся, что в случае двух классов выражение 2.13 превращается в 2.11. Имеем

$$P(y = 1|x) = \frac{e^{\zeta_1(x)}}{e^{\zeta_1(x)} + e^{\zeta_{-1}(x)}} = \frac{1}{1 + e^{\zeta_{-1}(x) - \zeta_1(x)}} = \sigma(\zeta_1(x) - \zeta_{-1}(x)).$$

При этом

$$\zeta_1(x) - \zeta_{-1}(x) = \ln P(1) - \ln P(-1) + \sum_{j=1}^n (u_j^1 - u_j^{-1}).$$

Заметив, что $u_j^1 - u_j^{-1} = \sigma^{-1}(g_j(x_j))$, получаем искомое.

2.4.3 Ряд Бахадура

Рассмотрим интересный подход, касающийся случая бинарных (двоичных) переменных.

Бинарными называются переменные, принимающие одно из двух значений: «истина» и «ложь», которые обозначаются соответственно 1 и 0.

Рассмотрим набор переменных $X_1, \dots, X_n$ со значениями $x = (x_1, \dots, x_n)$, $x_j \in \{0, 1\}$, $x \in X = \{0, 1\}^n$.

Если считать переменные независимыми, то для задания вероятностной модели требуется n параметров, а в общем случае модель совместного распределения имеет $2^n - 1$ параметров. Если объём выборки существенно больше 2^n , то оправдано оценивать совместное распределение, если объём выборки сравним с n , то

имеет смысл использовать модель независимых переменных, даже если нет оснований ожидать, что переменные действительно независимы.

В данном разделе рассмотрим модель, которую уместно использовать, когда объём выборки много больше n , но меньше 2^n . Данная модель связана с разложением совместного распределения в ряд Бахадура.

Идея разложения в ряд Бахадура состоит в том, чтобы, начиная с приближения независимых переменных, последовательно учитывать парные зависимости переменных, зависимости в тройках, четвёрках и т.д. Разложение в ряд Бахадура выглядит следующим образом

$$P(x) = P^1(x) \sum_{j=1}^n q_j(x).$$

Здесь $P^1(x)$ – значение совместной вероятности в предположении независимости переменных, т.е.

$$P^1(x) = \prod_{j=1}^n P(x_j).$$

Единица сверху — верхний индекс.

Остальные компоненты определяются как

$$\begin{aligned} q_1(x) &\equiv 1, & q_2(x) &= \sum_{j=1}^n \sum_{k=j+1}^n \rho_{jk} z_j z_k, \\ q_3(x) &= \sum_{j=1}^n \sum_{k=j+1}^n \sum_{l=k+1}^n \rho_{jkl} z_j z_k z_l, & \dots \end{aligned}$$

В этих выражениях использованы обозначения

$$z_j = \frac{x_j - p_j}{\sqrt{p_j(1 - p_j)}},$$

где $p_j = P(x_j = 1)$, а

$$\rho_{jk} = \sum_{x \in X} z_j z_k P(x), \quad \rho_{jkl} = \sum_{x \in X} z_j z_k z_l P(x), \quad \dots$$

Распределения

$$P^M(x) = P^1(x) \sum_{j=1}^M q_j(x).$$

представляют собой частичные суммы ряда Бахадура, при этом $P^2(x)$ учитывает только парные зависимости переменных, $P^3(x)$ учитывает парные зависимости и зависимости в тройках. Полная сумма ряда совпадает с исходным распределением $P(x)$.

Частичные суммы ряда Бахадура могут использоваться в качестве приближений для $P(x)$.

Чем больше членов ряда используется, тем больше параметров требуется оценить по выборке и тем выше сложность модели.

Следует выбирать сложность модели адекватной объёму выборки. Грубой рекомендацией здесь будет выбирать такую модель, чтобы число оцениваемых параметров было хотя бы в несколько раз меньше объёма выборки. Более точно выбрать сложность модели можно с помощью метода статистического моделирования.

Разложение в ряд Бахадура вполне может использоваться для решения задачи классификации в пространстве бинарных переменных.

Вместе с тем, подобные задачи встречаются довольно редко, поэтому описанное разложение относительно мало распространено на практике.

Тем не менее, знакомство с ним очень полезно для понимания механизмов, за счёт которых одни методы построения решающих функций оказываются эффективнее других. Мы ещё вернёмся к этому разложению при выяснении причин эффективности бустинга.

2.4.4 Свойства метода

Характеризуя «наивный» байесовский классификатор, можно сделать следующие замечания.

- Предположение независимости переменных эквивалентно аддитивности в пространстве преобразованных переменных.

- Полученное преобразование является разумным вариантом target encoding, может использоваться и для вещественных переменных.
- Наивный байесовский классификатор является частным случаем логистической регрессии с использованием target encoding.
- Предположение о (строгой) независимости переменных обычно оказывается неоправданно сильным, однако в ослабленном виде гипотеза независимости очень полезна.

Поскольку в реальных задачах предполагать независимость переменных как правило нет оснований, использовать «наивный» байесовский классификатор редко бывает целесообразным. Однако попробовать применить «ослабленный» вариант метода в виде логистической регрессии с target encoding — почти всегда оправдано. Если при этом коэффициенты регрессии окажутся близки к единице, то это может служить свидетельством независимости переменных (и тогда можно применить собственно наивный байесовский классификатор).

Отметим важный факт. Почти для всех методов машинного обучения имеет место правило, что чем больше переменных, тем больший объём выборки требуется для обучения (из-за чего даже появился термин «проклятие размерности»).

Исключением из этого правила является случай условно независимых переменных. Здесь всё наоборот: чем больше переменных, тем большую точность прогноза можно получить при том же объёме выборки. Это похоже на то, как привлечение множества независимых мнений позволяет сделать более точные выводы.

2.5 Метод опорных векторов

Метод был разработан ещё в 60-е годы (В.Н. Вапник и др.) и изначально назывался методом обобщённого портрета.

В 90-е годы метод был усовершенствован и стал называться машиной опорных векторов (support vector machine, SVM).